Trabajo Práctico Nº2

Software-Defined Networks

25/11/2025

2º cuatrimestre 2025

Redes (TA048)

Cátedra: López Pecora - Horn

Valentina Llanos Pontaut
104413
vllanos@fi.uba.ar

Milton Formiga
107744
mformiga@fi.uba.ar

María Agustina Fontana
108090
mafontana@fi.uba.ar

Santiago Novaro
110938
snovaro@fi.uba.ar

Franco Daniel Capra
99642
fcapra@fi.uba.ar

Índice

Introducción **2**
Implementación de la solución **2**
Topología en cadena 2
Controlador POX y firewall 3
Simulaciones y resultados **4**
Pruebas de conectividad sin restricciones 4
Pruebas de firewall 5
Bloqueo de tráfico con puerto destino 80 5
Bloqueo de tráfico UDP desde h1 al puerto 5001 6
Bloqueo de comunicación entre dos hosts específicos 7
Análisis global de las capturas y logs 8
Guía de ejecución (README) **8**
Archivos y estructura 8
Ejecución del controlador POX 9
Comando de ejecución 9
Parámetros y flags 9
Ejecución de la topología en Mininet 10
Comando de ejecución 10
Parámetros del script de topología 10
Flujo de ejecución recomendado 11
Preguntas teóricas **12**
Switch vs Router 12
Switch Convencional vs OpenFlow 12
Reemplazo por Switches OpenFlow 12
Conclusión **13**

Introducción

El presente trabajo práctico tiene como objetivo explorar el paradigma de redes definidas por software (SDN) utilizando el protocolo OpenFlow sobre un entorno emulado con Mininet y el controlador POX. A partir de una topología simple pero parametrizable, se busca comprender la separación entre plano de control y plano de datos, y cómo un controlador centralizado puede modificar dinámicamente el comportamiento de los switches de la red.

En particular, se implementa una topología en cadena compuesta por una cantidad variable de switches OpenFlow, con cuatro hosts ubicados en los extremos de la cadena. Sobre esta topología se desarrolla un controlador basado en POX que combina dos componentes: un módulo de learning switch de capa 2 y un módulo de firewall que aplica reglas de filtrado definidas externamente en un archivo JSON, las cuales son válidas únicamente en un switch seleccionado de la cadena.

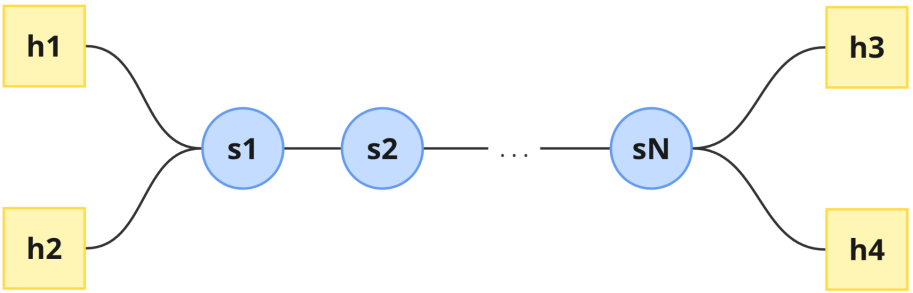
Para validar el funcionamiento de la solución se realizan distintas simulaciones y pruebas: conectividad básica mediante pingall, generación de tráfico TCP y UDP con iperf, captura y análisis de paquetes con Wireshark, y revisión de los logs del controlador POX. Estas herramientas permiten verificar tanto el comportamiento normal de la red como la correcta aplicación de las políticas de firewall solicitadas en el enunciado.

Implementación de la solución

Topología en cadena

La topología se implementó en Mininet mediante una clase ChainTopo, que construye una cadena de switches OpenFlow con cuatro hosts ubicados en los extremos, siguiendo el esquema pedido en el enunciado.

La estructura lógica es:



Las características principales son:

- La cantidad de switches de la cadena es flexible y se parametriza al ejecutar el script. Esto permite probar distintos escenarios simplemente variando este parámetro (por ejemplo, 2, 3, 5 switches, etc.)
- Los hosts se ubican de la siguiente manera:
    - h1 y h2 conectados al primer switch de la cadena
    - h3 y h4 conectados al último switch

- Cada switch se crea con un identificador (dpid) conocido y consistente con su índice en la cadena, de forma que el switch s1 tenga dpid = 1, s2 tenga dpid = 2, y así sucesivamente. Esto simplifica luego la selección del switch que actuará como firewall
- Todos los switches se configuran como Open vSwitch (OVS) y se conectan a un controlador remoto, que es el que decide cómo tratar el tráfico (aprendizaje L2, firewall, etc.)

De esta manera, la topología queda desacoplada del controlador: la misma cadena puede utilizarse tanto para pruebas sin firewall como para pruebas con distintas políticas de filtrado.

Controlador POX y firewall

El plano de control se implementó sobre el controlador POX, combinando dos módulos:

- Un módulo de switch de capa 2 con aprendizaje (learning switch) provisto por POX.
- Un módulo propio de firewall, desarrollado como ext.firewall.

El módulo de learning switch se encarga de que la red funcione como un switch Ethernet clásico, aprendiendo qué MAC se encuentra detrás de cada puerto y encaminando el tráfico en consecuencia. En ausencia de restricciones, esto permite que todos los hosts de la topología puedan comunicarse entre sí.

Sobre ese comportamiento básico, el módulo ext.firewall introduce una capa de políticas de filtrado:

- Las reglas de firewall no están codificadas “a mano” en el controlador, sino que se definen en un archivo JSON externo (**rules_file.json**). Esto permite modificar las políticas (por ejemplo, bloquear un puerto o un par de hosts) sin cambiar el código del controlador.
- Cada regla indica condiciones sobre el tráfico: direcciones IP, protocolo, puertos, etc. El firewall tendrá una entrada en la tabla de flujo por cada regla/patrón. Cuando un paquete que llega coincide con alguna regla, toma la acción definida.

El firewall no se aplica en toda la red, sino únicamente en un switch específico de la cadena. Ese switch se identifica por su **dpid**, que coincide con el índice de la cadena (por ejemplo, el switch central s2). De este modo, se respeta la consigna de que “el firewall lo tenga un único switch elegido por el grupo”, mientras el resto de la red solo aplica el comportamiento de learning switch.

En tiempo de ejecución, el flujo lógico es:

1. Cuando un switch se conecta al controlador, POX genera un evento **ConnectionUp**
2. El módulo ext.firewall verifica si el dpid de ese switch coincide con el dpid configurado como firewall
3. Si coincide, el controlador recorre todas las reglas del archivo JSON y, para cada una, construye y envía un mensaje **ofp_flow_mod** al switch

- Cada flujo matchea tráfico IPv4 (dl_type = 0x0800) y, opcionalmente, protocolo (nw_proto), IP de origen/destino (nw_src, nw_dst) y puertos (tp_src, tp_dst) según los campos presentes en la regla
- Los flujos se instalan sin acciones, lo que en OpenFlow 1.0 implica que todo paquete que matchee esa entrada se descarta directamente en el switch

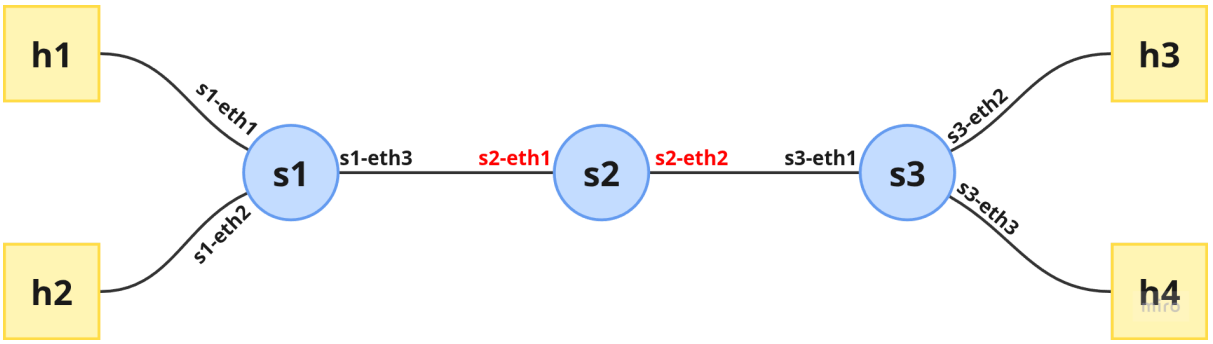
4. El módulo de learning switch (**forwarding.l2_learning**) continúa encargándose del resto del tráfico que no coincide con ninguna regla de firewall, manteniendo el comportamiento de switch L2 en el resto de la red

Con esta combinación, se logra una implementación en la que:

- La topología es parametrizable y reutilizable
- El comportamiento de red básica (aprendizaje L2) está centralizado en un módulo estándar de POX
- El firewall es configurable por archivo, se aplica solo en un switch de la cadena, y cumple las restricciones pedidas en el trabajo práctico

Simulaciones y resultados

Para validar el funcionamiento de la topología y del firewall se realizaron simulaciones sobre la red emulada en Mininet, controlada por POX. En todos los casos se levantó el controlador con el módulo de learning switch y el módulo de firewall, se inició la topología en cadena, se hicieron pruebas con pingall e iperf desde la CLI de Mininet y se analizaron capturas en Wireshark junto con los logs de POX para contrastar el comportamiento observado en la red con las reglas instaladas en el switch firewall.
En las pruebas con Wireshark se capturó tráfico sobre la interfaz **s2-eth1** y **s2-eth2**, que corresponden a los puertos del switch número 2 (s2), elegido como firewall:

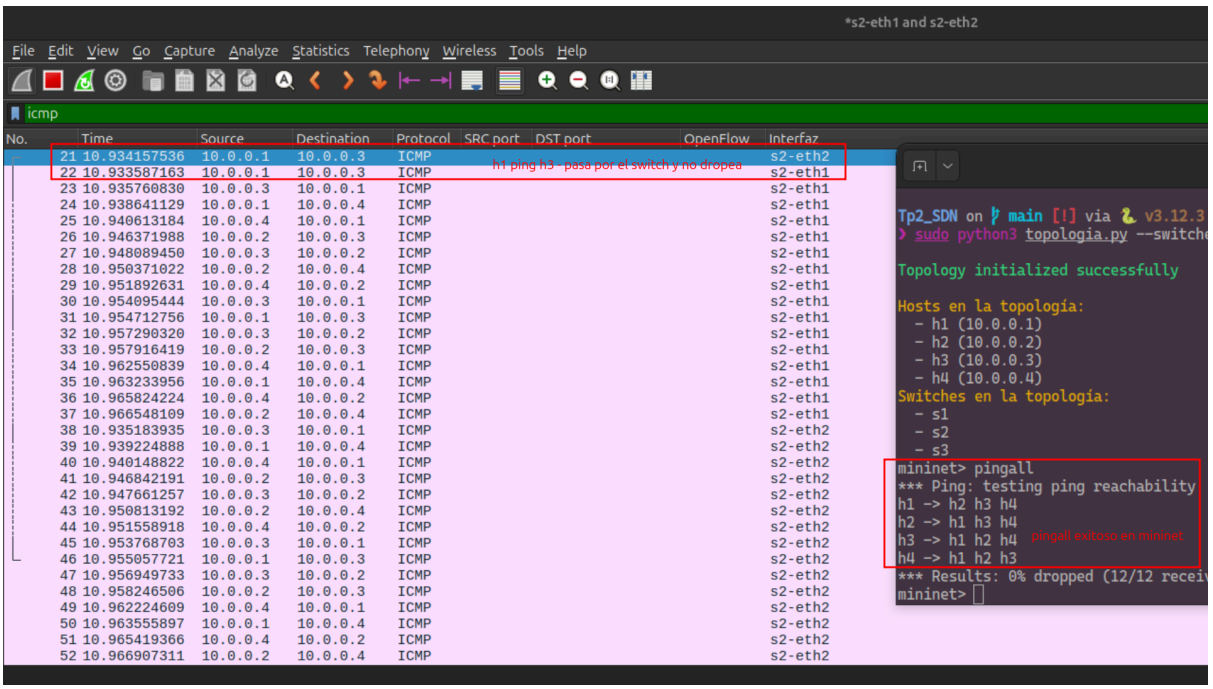


Pruebas de conectividad sin restricciones

Como primer paso se verificó el comportamiento de la topología sin reglas de firewall activas (archivo de reglas vacío). En este escenario la red se comporta como un conjunto de switches Ethernet con aprendizaje de capa 2:

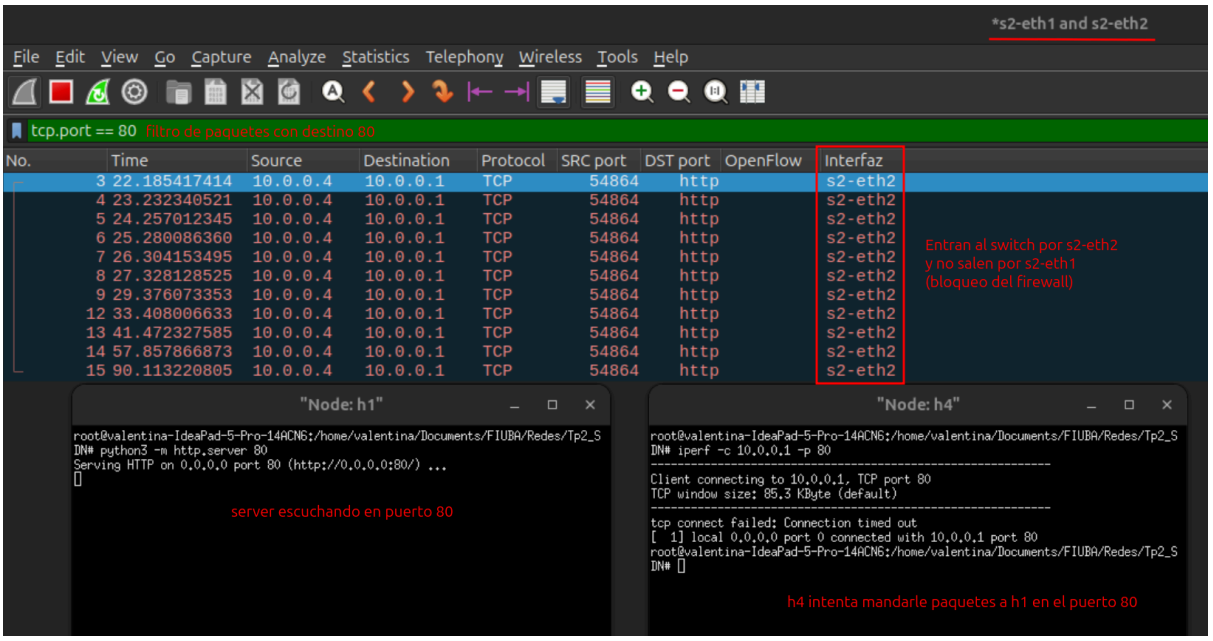
- Se ejecutó pingall para comprobar la conectividad entre todos los hosts (h1, h2, h3, h4).

Todos los ping fueron exitosos (0 % de pérdida). Estas pruebas sirven como línea de base para comparar luego el efecto de las reglas del firewall.

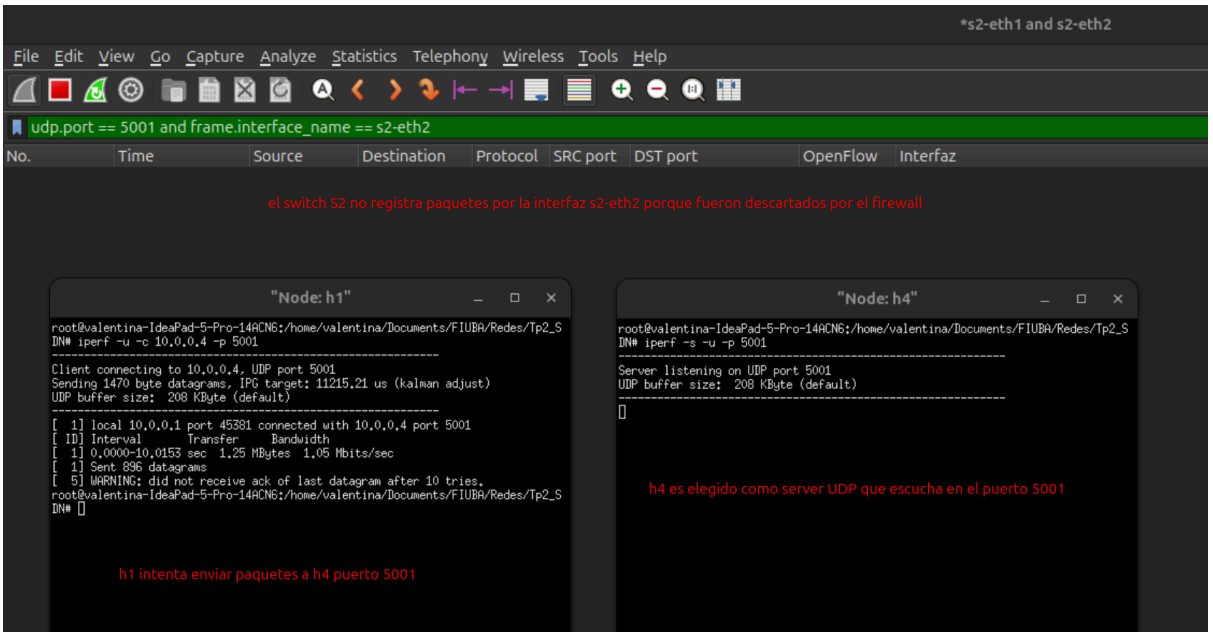


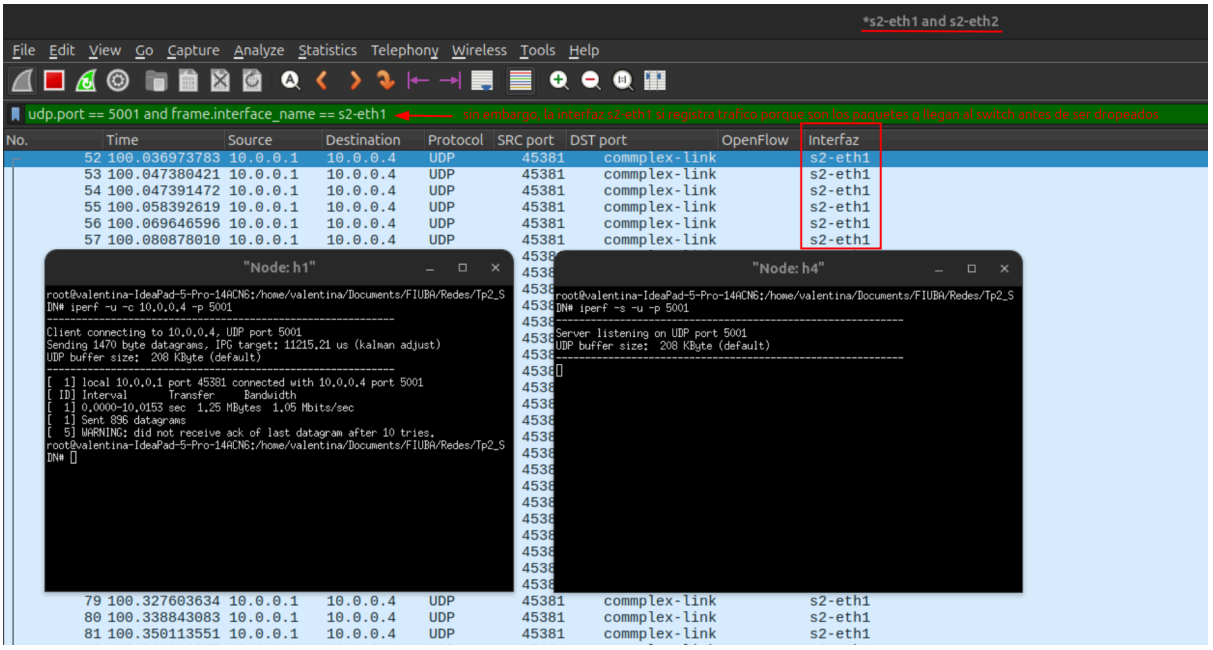
Pruebas de firewall

Bloqueo de tráfico con puerto destino 80

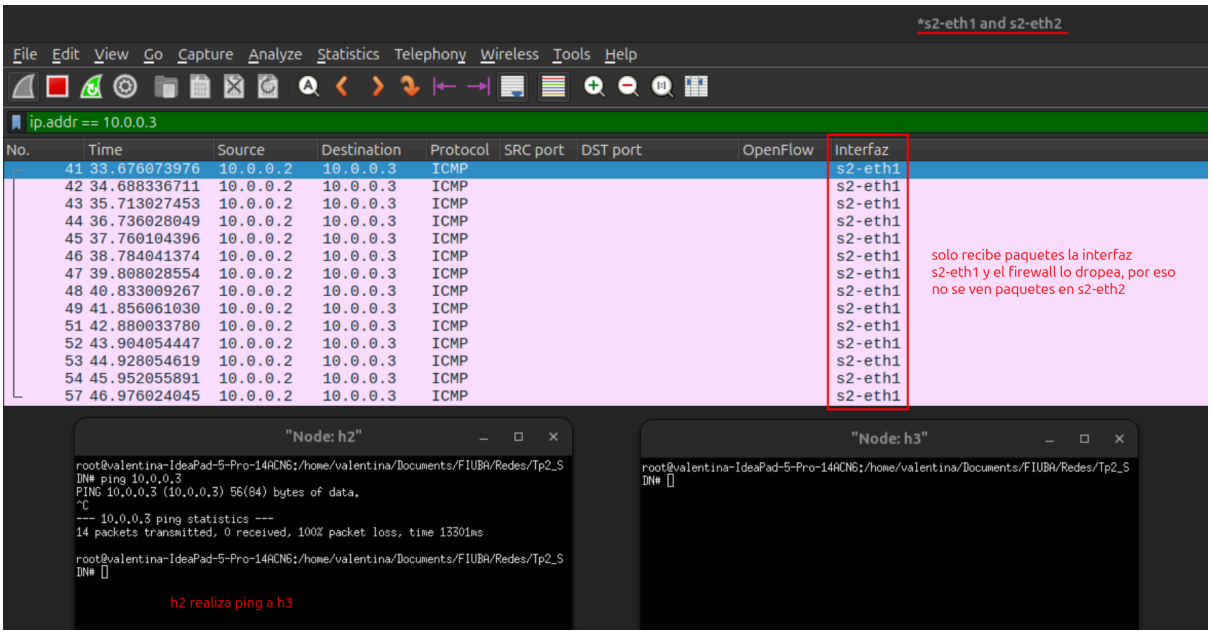


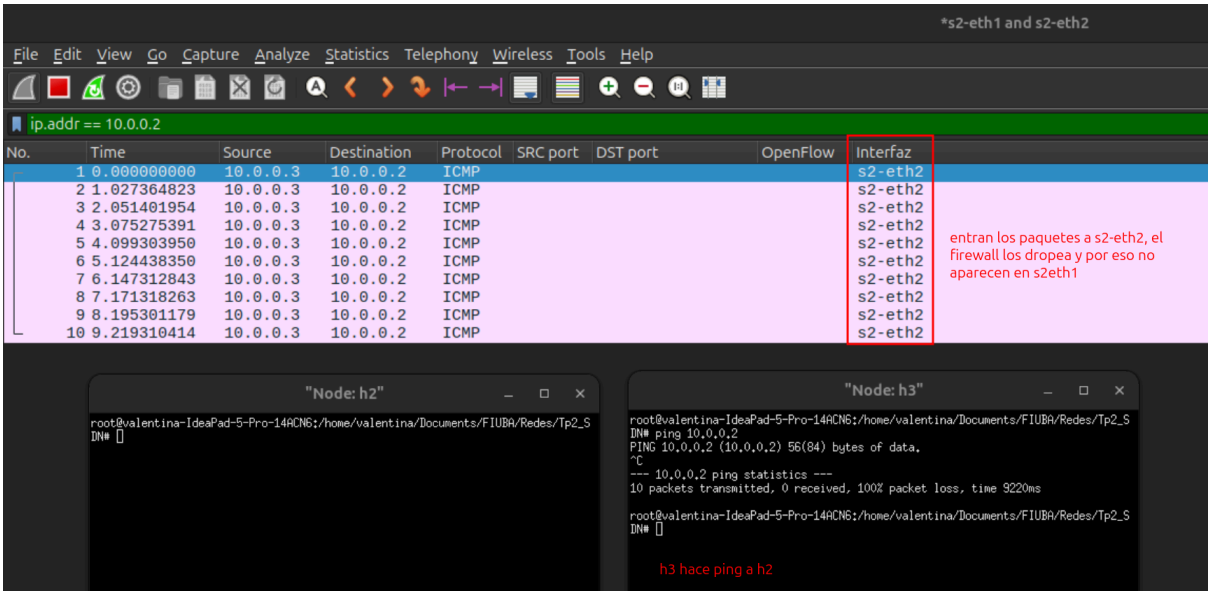
Bloqueo de tráfico UDP desde h1 al puerto 5001





Bloqueo de comunicación entre dos hosts específicos





pingall con mininet:

```
Hosts en la topología:
  - h1 (10.0.0.1)
  - h2 (10.0.0.2)
  - h3 (10.0.0.3)
  - h4 (10.0.0.4)
Switches en la topología:
  - s1
  - s2
  - s3
  - s4
  - s5
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 X h4
h3 -> h1 X h4
h4 -> h1 h2 h3
*** Results: 16% dropped (10/12 received)
mininet>
```

Análisis global de las capturas y logs

Las capturas de Wireshark son consistentes con el comportamiento esperado del firewall en el switch de la cadena. Sin reglas definidas, las capturas muestran el intercambio normal de paquetes entre todos los hosts y los logs se limitan a los mensajes de conexión de los switches y al funcionamiento del módulo de learning switch.
Al activar las reglas, el tráfico que coincide con los patrones del archivo JSON deja de aparecer más allá del switch firewall: en la prueba de puerto 80 no se observa tráfico de aplicación hacia ese puerto en los hosts destino y, en la prueba UDP desde h1 al puerto 5001, los paquetes nunca llegan a s2-eth2.
En los casos donde el tráfico no coincide con ninguna regla, las capturas muestran que los paquetes se encaminan normalmente entre los hosts y las pruebas de ping e iperf son exitosas. Esto confirma que el módulo de firewall solo afecta a los flujos que cumplen las condiciones especificadas en el archivo de reglas, mientras que el resto del tráfico es manejado por el módulo de learning switch sin restricciones.

Guía de ejecución (README)

Esta sección describe cómo levantar el controlador POX con firewall y la topología en Mininet, indicando los parámetros de ejecución, sus valores por defecto y el orden recomendado de ejecución

Archivos y estructura

Se cuenta con la siguiente estructura:

```
~/pox/
  └─ pox/
     └─ ext/
```

```
        ├─ firewall.py        # Módulo del firewall (controlador)
        └─ rules_file.json    # Archivo de reglas del firewall
~/topologia.py                # Script de Mininet con la topología
```

Ejecución del controlador POX

El controlador se basa en dos módulos:

- **`ext.firewall`:** aplica las reglas de firewall sobre un único switch (identificado por su dpid)
- **`forwarding.l2_learning`:** implementa el comportamiento de switch de capa 2 con aprendizaje (L2 learning)

Comando de ejecución

```
./pox/pox.py log.level --DEBUG ext.firewall --rules=rules_file.json
--dpid=2 forwarding.l2_learning
```

Parámetros y flags

A continuación se describen los parámetros de este comando:

`log.level --DEBUG`

- Obligatorio: no
- Descripción: configura el nivel de log global de POX
- Valor por defecto: si no se indica, el nivel típico es INFO
- Uso recomendado: `--DEBUG` para obtener más detalle en los logs y poder contrastarlos con las capturas de Wireshark

`ext.firewall`

- Obligatorio: si
- Descripción: módulo de POX ubicado en pox/ext/firewall.py
    - Carga las reglas desde el archivo JSON indicado por --rules
    - Escucha el evento **ConnectionUp** de los switches
    - Si el dpid del switch coincide con el configurado por --dpid, instala en ese switch las reglas de firewall como entradas de flujo OpenFlow (ofp_flow_mod)
    - Cada regla del JSON se traduce en un patrón de match (tráfico IPv4, protocolo, IPs y/o puertos).

`--rules=rules_file.json`

- Obligatorio: no (pero requerido por la consigna del TP)
- Descripción: nombre del archivo JSON que contiene las reglas del firewall
- Ámbito: se interpreta relativo al directorio del módulo (pox/ext/)
- Valor por defecto: `rules_file.json` (según la firma de launch en firewall.py)

`--dpid=2`

- Obligatorio: no, pero es la forma explícita de indicar qué switch actuará como firewall
- Descripción: identifica el switch (por su dpid) sobre el que se aplicarán las reglas de firewall
- Valor por defecto: 1 (switch con dpid=1, típicamente s1), según el valor por defecto del parámetro en launch

`forwarding.l2_learning`

- Obligatorio: no desde el punto de vista de POX, pero sí en este diseño
- Descripción: módulo estándar de POX que:
  - Aprende qué MAC está por qué puerto
  - Instala reglas de forwarding (flow entries) en los switches
  - Interacción con el firewall:
    - ext.firewall instala en el switch firewall reglas OpenFlow con prioridad más alta para los flujos que deben ser filtrados (ALLOW/DENY)
    - Para el tráfico que no matchea ninguna regla de firewall, `forwarding.l2_learning` se encarga de manejarlo como un switch L2 con aprendizaje

Ejecución de la topología en Mininet

El archivo **`topologia.py`** define la clase ChainTopo y un main que permite parametrizar la topología y el controlador remoto.

Comando de ejecución

En otra terminal (mientras POX está corriendo), desde el directorio donde se encuentre topologia.py:

```
sudo python3 topologia.py --switches 3 --ctrl-ip 127.0.0.1 --ctrl-port 6633
```

Se pueden omitir parámetros que se deseen dejar con su valor por defecto

Parámetros del script de topología

`--switches`

- Obligatorio: no
- Tipo: entero
- Descripción: cantidad de switches en la cadena
- Valor por defecto: 3
- Efecto: crea la topología:
  - Hosts:
    - h1 (10.0.0.1/24) y h2 (10.0.0.2/24) conectados al primer switch (s1)
    - h3 (10.0.0.3/24) y h4 (10.0.0.4/24) conectados al último switch (sN)
  - Switches:
    - s1, s2, …, sN conectados en cadena
    - Cada switch se crea con dpid=f"{i:016x}", de modo que:
      - s1 → dpid = 1
      - s2 → dpid = 2
      - s3 → dpid = 3

- etc.

`--info`

- Obligatorio: no
- Tipo: flag (sin valor)
- Descripción: si se incluye, muestra:
  - Configuración de interfaces (ifconfig) de cada host
  - Configuración de colas (tc qdisc show) de hosts y switches
- Valor por defecto: False (no se muestra información extra)

`--ctrl-ip`

- Obligatorio: no
- Tipo: string (dirección IP)
- Descripción: IP del controlador remoto
- Valor por defecto: 127.0.0.1
- Requisito: debe coincidir con la IP donde se está ejecutando POX (si todo corre en la misma VM/máquina, se mantiene en 127.0.0.1)

`--ctrl-port`

- Obligatorio: no
- Tipo: entero
- Descripción: puerto TCP donde escucha el controlador remoto
- Valor por defecto: 6633 (puerto clásico para OpenFlow/POX)
- Requisito: debe coincidir con el puerto configurado en POX (por defecto también 6633)

Flujo de ejecución recomendado

1. Iniciar POX con el firewall y L2 learning

```
./pox/pox.py log.level --DEBUG ext.firewall --rules=rules_file.json
--dpid=2 forwarding.l2_learning
```

2. Iniciar la topología en Mininet

```
sudo python3 topologia.py --switches 3 --ctrl-ip 127.0.0.1 --ctrl-port 6633
```

3. Realizar pruebas desde el CLI de Mininet

```
mininet> pingall
mininet> iperf h1 h3
```

4. Finalizar

- Salir del CLI de Mininet con exit
- Detener POX con Ctrl+C

Preguntas teóricas

Switch vs Router

"¿Cuál es la diferencia entre un Switch y un router? ¿Qué tienen en común?"

Diferencias:
Los switches tradicionales operan principalmente en la capa de enlace, tomando decisiones de reenvío basadas en direcciones MAC. Mientras que los routers operan en la capa de red, tomando decisiones de ruteo basadas en direcciones IP para interconectar diferentes redes.
El switch utiliza la tabla CAM (Content Addressable Memory) para asociar las MACs a puertos físicos. El router utiliza tablas de ruteo para determinar el mejor camino hacia una red destino.
Similitudes:
Ambos tienen su arquitectura dividida en dos planos: el plano de control (donde se toman las decisiones sobre cómo administrar el dispositivo, generalmente sobre software) y el plano de datos (donde se envían los paquetes, sobre hardware).
Ambos dispositivos son intermediarios que conmutan paquetes para permitir la interconexión de computadoras.

Switch Convencional vs OpenFlow

"¿Cuál es la diferencia entre un Switch convencional y un Switch OpenFlow?"

Switch convencional:
El plano de control y de datos funcionan en el mismo hardware, creando decisiones de reenvío internamente.
Se rutea solamente según la MAC de destino, lo cual es insuficiente para las demandas modernas.
Switch OpenFlow:
El plano de control se mueve a un software controlador externo, dejando al switch como un dispositivo que sólo ejecuta instrucciones.
El controlador externo define las reglas de las tablas de flujo del switch a través del protocolo OpenFlow, que permite generar políticas basadas en flujos utilizando hasta 10 campos de los headers, no solamente la MAC de origen y destino.
Es más flexible, ya que permite programar el comportamiento directamente sobre las tablas de flujo del hardware en tiempo real.

Reemplazo por Switches OpenFlow

"¿Se pueden reemplazar todos los routers de la Internet por Switches OpenFlow? Piense en el escenario interASes para elaborar su respuesta"

No, no se pueden reemplazar todos los routers de Internet por switches OpenFlow. El escenario interAses (entre Sistemas Autónomos) es un ejemplo claro.

Los routers ASBR (routers de borde de sistema autónomo) intercambian información de rutas ejecutando BGP (border gateway protocol). Este protocolo es fundamental para lograr la comunicación interAses permitiendo que cada AS pueda mantener políticas de ruteo propias, negociar rutas con proveedores, etc y que funcione la estructura global de Internet.
Para poder reemplazar estos routers tradicionales, necesitarías un controlador OpenFlow capaz de ejecutar una implementación completa de BGP, manejar políticas interAs complejas y traducirlas a reglas OpenFlow. Supera el modelo para el que OpenFlow fue diseñado, que fue pensado para dominios más acotados.
Además, si se desarrollara esa implementación, integrando BGP a un único controlador SDN que manejara la red global de manera centralizada, sería incompatible con la arquitectura descentralizada de Internet. Las consecuencias incluirían que ninguna red tendría control sobre su propio ruteo, perdiendo la autonomía de las AS.
Por estas razones, no es viable reemplazar los routers de Internet por switches OpenFlow.

Conclusión

El presente Trabajo Práctico permitió alcanzar el objetivo principal de explorar de manera práctica el paradigma de Redes Definidas por Software (SDN) a través de la implementación de una solución con el protocolo **OpenFlow** en un entorno emulado con **Mininet** y el controlador **POX**.

Los resultados de las simulaciones con **pingall**, **iperf** y el análisis de tráfico con **Wireshark** y logs de **POX** validaron el comportamiento esperado:

- En ausencia de reglas, la red opera como un switch Ethernet tradicional.
- Al activar las reglas configurables en el archivo JSON, el tráfico coincidente fue descartado correctamente por el único switch designado como firewall (s2 en el ejemplo), mientras que el tráfico no afectado continuó siendo gestionado por el módulo de learning switch.

En síntesis, este trabajo práctico ofreció una comprensión sólida de cómo SDN permite una gestión de red más programable, eficiente y centralizada, cumpliendo con todos los requerimientos planteados en el enunciado.